CSE MASTER ROADMAP — SEMESTER III TO VIII

Part A = DSA | Part B = Core CS / Development

PRINT FORMAT: Times New Roman • Bold headings • Spacious checkboxes • Follow the phases in order, but keep old DSA topics in weekly revision.

1. THE CORRECT LOGIC

- ☐ C++ foundation comes before serious DSA.
- ☐ Time Complexity and Space Complexity are learned at the beginning of DSA, before advanced problem solving.
- ☐ Basic DSA comes before searching, sorting, trees and graphs.
- ☐ Computer fundamentals come before deeper OS concepts.
- ☐ DBMS starts early and grows in depth gradually.
- ☐ OS starts after basic computer/process concepts.
- ☐ CN starts after the basic computer foundation and while DSA continues.
- ☐ Advanced DSA comes after recursion, trees, graphs and core problem-solving patterns.
- ☐ Development and system design come later, after DSA + DBMS + OS + CN foundations.
- ☐ DSA NEVER STOPS: every phase contains a revision/practice loop.
- ☐ C++ is the primary interview language: solve every DSA problem in C++ from day one.
- ☐ Placement preparation begins in III semester, not in the final year.
- ☐ CIE and SEE exam periods are built into the schedule; DSA slows down during exams but never stops.
- ☐ Development follows a concrete stack: HTML/CSS/JS, then React, then Node.js + Express + MySQL.

2. MASTER FLOW

- ☐ C++ Foundation
- ☐ PHASE 1 — DSA Foundations + Computer Fundamentals
- ☐ PHASE 2 — Basic Data Structures + DBMS Foundation
- ☐ PHASE 3 — Searching/Sorting + OS Foundation
- ☐ PHASE 4 — Recursion/Backtracking + DBMS Advanced
- ☐ PHASE 5 — Trees/Heaps + Computer Networks Foundation
- ☐ PHASE 6 — Graphs + OS/CN Advanced
- ☐ PHASE 7 — Greedy/DP + Core CS Consolidation
- ☐ PHASE 8 — Advanced DSA + Computer Science Advanced Topics
- ☐ PHASE 9 — Software Design + Development
- ☐ PHASE 10 — Projects + System Design + Security
- ☐ PHASE 11 — Internship / Placement Preparation

3. SEMESTER TIMELINE (III TO VIII)

Odd semesters run roughly Aug–Dec and even semesters Feb–Jun. Adjust to your college calendar. Phases map to semesters as follows:

Sem	Phases Covered	Development Track	Key Deliverable
III	Phase 0, 1, 2 (C++, complexity, basic DS, DBMS foundation)	HTML, CSS, JavaScript basics; Git and GitHub	~100 problems; GitHub, LeetCode profiles; resume draft
IV	Phase 3, 4 (searching/sorting, OS foundation, recursion/backtracking, DBMS advanced)	Advanced JavaScript; React	~120 problems; Project 1 (React app)
V	Phase 5, 6 (trees/heaps, CN, graphs, OS/CN advanced)	Node.js, Express, MySQL, JWT auth	Tier 1 complete; Project 2 (full-stack with auth)
VI	Phase 7, 8 (selective), 9 (greedy/DP, advanced DSA, software design)	Deployment, testing, design patterns, UML	Project 3 deployed; internship secured/applied
VII	Phase 10, 11 begins (projects, system design, security, placement)	System design basics, security, project polish	Interview-ready; applications submitted
VIII	Phase 11 continues (offers, off-campus, maintenance)	Final-year project; maintenance	Offer confirmed; final project and viva complete

4. DSA PRIORITY TIERS AND PRIMARY LANGUAGE

Primary Interview Language: C++

- Use C++ for every problem from day one; do not switch languages mid-way.
- Learn the STL early: vector, string, pair, map, unordered_map, set, stack, queue, deque, priority_queue, sort with lambdas, lower_bound/upper_bound.
- Know STL complexities (map is O(log n), unordered_map is O(1) average).
- Keep a personal template file with fast I/O, typedefs and helper functions.

Tier 1 / Tier 2 / Tier 3

Tier	Topics	Goal
Tier 1 (Must master)	Arrays, strings, hashing, two pointers, sliding window, binary search, sorting, linked list, stack/queue, recursion and backtracking, trees and BST, heaps, BFS/DFS, basic greedy, 1D/2D DP	Solve medium problems in 25–30 min without hints; covers most service and product-company rounds
Tier 2 (Should know)	Knapsack, LCS, LIS, partition and grid DP, Dijkstra, Bellman-Ford, topological sort, union-find, Prim/Kruskal, trie, monotonic stack, intervals, two heaps, bit manipulation, LRU cache	Solve most mediums and some hards; needed for good product companies
Tier 3 (Selective)	Segment tree, Fenwick tree, KMP/Rabin-Karp/Boyer-Moore, AVL, bitmask and digit DP, SCC and bridges, A*, Ford-Fulkerson, TSP, B-trees	Only after Tier 1 and 2 are solid; targets top-tier rounds. Other advanced items in Phase 8 are awareness-only.

5. PHASE 0 — C++ PROGRAMMING FOUNDATION

When: III Semester, weeks 1–3 **Focus:** Foundation. All tiers depend on this.

Part A — Programming Foundation for DSA

- ☐ C++ syntax
- ☐ Variables and data types
- ☐ Input/output
- ☐ Operators
- ☐ Type conversion
- ☐ Control structures
- ☐ if/else
- ☐ switch
- ☐ for
- ☐ while
- ☐ do-while
- ☐ Functions
- ☐ Parameters and return values
- ☐ Pass by value
- ☐ Pass by reference
- ☐ Pseudocode
- ☐ OOP basics
- ☐ Arrays
- ☐ Strings
- ☐ Pointers
- ☐ References
- ☐ Basic recursion
- ☐ STL basics

Part B — Supporting Computer Fundamentals

- ☐ Basic idea of CPU, memory and programs
- ☐ Binary/decimal number awareness
- ☐ Basic bitwise operations

Checkpoint

- ☐ Write C++ programs without copying
- ☐ Use functions and arrays comfortably
- ☐ Understand pointers/references
- ☐ Use vector, string, pair, stack, queue, map/unordered_map

6. PHASE 1 — DSA FOUNDATIONS + COMPUTER FUNDAMENTALS

When: III Semester, weeks 3–6 **Focus:** Foundation. Complexity analysis is used in every later problem.

Part A — DSA: LEARN COMPLEXITY FIRST

- ☐ What are data structures?
- ☐ Why data structures are important

- ☐ Time Complexity
- ☐ Space Complexity
- ☐ Time vs Space trade-off
- ☐ How to calculate complexity
- ☐ Asymptotic notation
- ☐ Big-O notation
- ☐ Big-Theta notation
- ☐ Big-Omega notation
- ☐ Constant O(1)
- ☐ Logarithmic O(log n)
- ☐ Linear O(n)
- ☐ Polynomial O(n^2 etc.)
- ☐ Exponential
- ☐ Factorial
- ☐ Analyze loops
- ☐ Analyze nested loops
- ☐ Analyze simple recursive code

DSA RULE: Do not rush past complexity. You should be able to look at code and estimate its time and space complexity before moving ahead.

Part B — How Computers Work

- ☐ How CPU executes programs
- ☐ Registers and RAM
- ☐ Instructions and programs
- ☐ CPU cache
- ☐ How computers calculate
- ☐ Unicode
- ☐ ASCII
- ☐ Character encodings
- ☐ Endianness
- ☐ Big Endian
- ☐ Little Endian
- ☐ Bitwise operators
- ☐ Floating point math

Retention

- ☐ 5 simple complexity questions
- ☐ 2 old C++ problems
- ☐ 1 weekly review of O(1), O(log n), O(n), O(n^2)

7. PHASE 2 — BASIC DATA STRUCTURES + DBMS FOUNDATION

When: III Semester, weeks 7 to SEE **Focus:** Tier 1: arrays, linked list, stack/queue, hashing. Start HTML/CSS/JS and Git alongside.

Part A — DSA

- ☐ Array
- ☐ Linked List
- ☐ Singly Linked List
- ☐ Doubly Linked List
- ☐ Circular Linked List
- ☐ Stack
- ☐ Queue
- ☐ Hash Table
- ☐ Hashing basics
- ☐ Implement basic structures from scratch

Part B — DBMS

- ☐ What is a database?
- ☐ SQL vs NoSQL
- ☐ Relational model
- ☐ Tables, rows and columns
- ☐ Primary key
- ☐ Foreign key
- ☐ Candidate key
- ☐ Composite key
- ☐ Constraints
- ☐ Entity-Relationship Model
- ☐ Basic SQL
- ☐ DDL
- ☐ DML
- ☐ DQL
- ☐ DCL
- ☐ SELECT
- ☐ WHERE
- ☐ ORDER BY
- ☐ GROUP BY
- ☐ HAVING
- ☐ DISTINCT
- ☐ INSERT
- ☐ UPDATE
- ☐ DELETE

☐ JOINs

Retention

☐ 2 array problems
☐ 2 linked-list problems
☐ 1 stack/queue problem
☐ 1 hashing problem
☐ 5 SQL queries
☐ Complexity analysis of each DSA solution

8. PHASE 3 — SEARCHING/SORTING + OS FOUNDATION

> **When:** IV Semester, first half **Focus:** Tier 1: binary search, merge sort, quick sort. Learn heap sort again in Phase 5.

Part A — DSA

☐ Linear Search
☐ Binary Search
☐ Binary-search variations
☐ Bubble Sort
☐ Selection Sort
☐ Insertion Sort
☐ Merge Sort
☐ Quick Sort
☐ Heap Sort
☐ Stable vs unstable sorting
☐ In-place vs non-in-place sorting
☐ Compare sorting complexities

Part B — Operating Systems Foundation

☐ What is an Operating System?
☐ Processes
☐ Process vs program
☐ Process states
☐ Processes and Threads
☐ Processes vs Threads
☐ Scheduling Algorithms
☐ CPU scheduling basics
☐ CPU Interrupts
☐ Basic process scheduling problems

Retention

☐ 2 binary-search problems
☐ 2 sorting problems

☐ 2 old array/linked-list problems

☐ 1 OS revision session

9. PHASE 4 — RECURSION/BACKTRACKING + DBMS ADVANCED

> **When:** IV Semester, second half **Focus:** Tier 1: recursion, subsets, permutations, combinations, N-Queens. Tier 3 (awareness only): Hamiltonian paths, Knight's Tour.

Part A — DSA

☐ Recursion

☐ Base case

☐ Recursive case

☐ Tail recursion

☐ Non-tail recursion

☐ Recursion tree

☐ Backtracking

☐ Subsets

☐ Permutations

☐ Combinations

☐ N-Queens

☐ Maze solving

☐ Hamiltonian paths

☐ Knight's Tour

Part B — DBMS

☐ Normalization / Denormalization

☐ Functional dependency basics

☐ 1NF

☐ 2NF

☐ 3NF

☐ BCNF

☐ Transactions

☐ ACID model

☐ BASE

☐ Locking

☐ Indexes

☐ Views

☐ Stored Procedures

☐ Database Federation

☐ Replication

☐ Sharding

☐ CAP Theorem

☐ PACELC

Retention

- ☐ 2 recursion problems
- ☐ 2 backtracking problems
- ☐ 2 SQL problems
- ☐ 1 normalization problem
- ☐ 1 transaction/ACID revision

10. PHASE 5 — TREES/HEAPS + COMPUTER NETWORKS FOUNDATION

> **When:** V Semester, first half **Focus:** Tier 1: trees, BST, traversals, heaps, priority queue.

Part A — DSA

- ☐ Tree terminology
- ☐ Binary Tree
- ☐ Full Binary Tree
- ☐ Complete Binary Tree
- ☐ Balanced Tree
- ☐ Unbalanced Tree
- ☐ Binary Search Tree
- ☐ Tree traversal
- ☐ Pre-order
- ☐ In-order
- ☐ Post-order
- ☐ Breadth First Search on trees
- ☐ Depth First Search on trees
- ☐ Tree search algorithms
- ☐ Heap
- ☐ Min Heap
- ☐ Max Heap
- ☐ Priority Queue

Part B — Computer Networks

- ☐ What is networking?
- ☐ Networking basics
- ☐ OSI Model
- ☐ TCP/IP Model
- ☐ DNS
- ☐ HTTP
- ☐ TLS & HTTPS
- ☐ Sockets
- ☐ IP address basics

☐ TCP

☐ UDP

Retention

☐ 2 tree traversal problems

☐ 2 BST problems

☐ 1 heap problem

☐ 1 SQL revision

☐ OSI/TCP-IP revision

11. PHASE 6 — GRAPHS + OS/CN ADVANCED

> **When:** V Semester, second half **Focus:** Tier 1: graph representation, BFS, DFS, cycle detection, islands. Tier 2: Dijkstra, Prim, Kruskal, Bellman-Ford. Tier 3: A*, Ford-Fulkerson.

Part A — DSA

☐ Graph

☐ Directed Graph

☐ Undirected Graph

☐ Spanning Tree

☐ Graph representation

☐ Adjacency List

☐ Adjacency Matrix

☐ Breadth First Search

☐ Depth First Search

☐ Cycle detection

☐ Shortest Path Algorithms

☐ Dijkstra's Algorithm

☐ Bellman-Ford Algorithm

☐ A* Algorithm

☐ Minimum Spanning Tree

☐ Prim's Algorithm

☐ Kruskal's Algorithm

☐ Island Traversal

☐ Ford-Fulkerson Algorithm

Part B — OS + CN Advanced

☐ Memory Management

☐ Lock

☐ Mutex

☐ Semaphore

☐ Concurrency in Multiple Cores

☐ Deadlock concepts

- ☐ TCP reliability
- ☐ TCP vs UDP
- ☐ DNS working
- ☐ HTTP request/response
- ☐ TLS/HTTPS
- ☐ Sockets
- ☐ Routing basics
- ☐ Subnetting basics

Retention

- ☐ 3 graph problems
- ☐ 2 old tree problems
- ☐ 1 greedy-style graph problem
- ☐ OS synchronization revision
- ☐ CN protocol revision

12. PHASE 7 — GREEDY + DYNAMIC PROGRAMMING + CORE CS CONSOLIDATION

> **When:** VI Semester, first half **Focus:** Tier 1: two pointers, sliding window, basic greedy, 1D/2D DP. Tier 2: knapsack, intervals, two heaps, cyclic sort, Huffman. Tier 3: randomised algorithms, longest path.

Part A — DSA

- ☐ Brute Force
- ☐ Greedy Algorithms
- ☐ Dijkstra's Algorithm
- ☐ Huffman Coding
- ☐ Kruskal's Algorithm
- ☐ Prim's Algorithm
- ☐ Dynamic Programming
- ☐ Optimal substructure
- ☐ Overlapping subproblems
- ☐ Memoization
- ☐ Tabulation
- ☐ 1D DP
- ☐ 2D DP
- ☐ Knapsack Problem
- ☐ Longest Path Problem
- ☐ Divide and Conquer
- ☐ Two Pointer Technique
- ☐ Sliding Window Technique
- ☐ Fast and Slow Pointers
- ☐ Merge Intervals

☐ Cyclic Sort
☐ Two Heaps
☐ Kth Element
☐ Randomised Algorithms

Part B — Core CS Consolidation

☐ DBMS revision
☐ OS revision
☐ CN revision
☐ SQL revision
☐ ACID/transactions
☐ Processes/threads
☐ Scheduling
☐ Memory management
☐ OSI/TCP-IP
☐ HTTP/DNS/TLS

Retention

☐ 2 greedy problems
☐ 2 DP problems
☐ 2 mixed old DSA problems
☐ 1 DBMS + OS + CN mixed revision

13. PHASE 8 — ADVANCED DSA + ADVANCED COMPUTER SCIENCE

When: VI Semester, second half (selective) **Focus:** Tier 2: Trie, Union-Find, LRU cache. Tier 3: segment tree, Fenwick, KMP, Rabin-Karp, Boyer-Moore, AVL. Everything else (Red-Black, 2-3, B-trees, K-D, ISAM, TSP, NP topics) is read-for-awareness only.

Part A — Advanced DSA

☐ Trie
☐ Segment Tree
☐ Fenwick Tree
☐ Disjoint Set / Union-Find
☐ Suffix Trees and Arrays
☐ AVL Trees
☐ Red-Black Trees
☐ 2-3 Search Trees
☐ 2-3-4 Trees
☐ K-ary / M-ary Trees
☐ B-Tree
☐ B/B+ Trees
☐ K-D Trees
☐ Skip Lists

- ☐ ISAM
- ☐ Advanced tree-based indexing
- ☐ Linear indexing
- ☐ Search Pattern in Text
- ☐ Suffix Arrays
- ☐ String Search and Manipulations
- ☐ Substring Search
- ☐ Brute Force Search
- ☐ Rabin-Karp
- ☐ Knuth-Morris-Pratt
- ☐ Boyer-Moore
- ☐ Travelling Salesman Problem
- ☐ Complex graph/optimization problems

Part B — Advanced Computer Science

- ☐ Probability
- ☐ Combinatorics
- ☐ P
- ☐ NP
- ☐ P = NP concept
- ☐ Co-NP
- ☐ NP-Hard
- ☐ NP-Complete
- ☐ Caches
- ☐ LRU Cache
- ☐ LFU Cache
- ☐ MFU Cache
- ☐ Process Forking
- ☐ Advanced memory management
- ☐ Advanced concurrency concepts

14. PHASE 9 — SOFTWARE DESIGN + DEVELOPMENT

> **When:** VI Semester, alongside Phase 8 **Focus:** DSA maintenance. Follow the concrete React + Node.js + Express + MySQL path in Section 17.

Part A — DSA Maintenance

- ☐ 3–5 mixed DSA problems per week
- ☐ Revisit weak topics
- ☐ Re-solve previously failed problems
- ☐ Maintain complexity analysis

Part B — Software Engineering / Design / Development

- ☐ Git
- ☐ GitHub
- ☐ Version control
- ☐ Common UML diagrams
- ☐ Class diagrams
- ☐ Use Case diagrams
- ☐ Activity diagrams
- ☐ State Machine diagrams
- ☐ Sequence diagrams
- ☐ GoF Design Patterns
- ☐ Architectural Patterns
- ☐ Dependency Injection
- ☐ Null Object Pattern
- ☐ Type Object Pattern
- ☐ Clean code
- ☐ Testing
- ☐ Debugging
- ☐ Backend fundamentals
- ☐ HTTP
- ☐ REST APIs
- ☐ JSON
- ☐ Authentication
- ☐ Authorization
- ☐ CRUD
- ☐ Database integration
- ☐ API testing

15. PHASE 10 — PROJECTS + SYSTEM DESIGN + SECURITY

> **When:** VII Semester, first half **Focus:** DSA maintenance plus company-wise sets. Finish Project 3 and learn system design basics.

Part A — DSA + Core Revision

- ☐ Weekly DSA maintenance
- ☐ Monthly mixed DSA set
- ☐ DBMS interview revision
- ☐ OS interview revision
- ☐ CN interview revision
- ☐ OOP revision

Part B — Projects / System Design / Security

- ☐ Small project

- ☐ Full-stack/backend project
- ☐ Real-world resume project
- ☐ Deployment
- ☐ Documentation
- ☐ Scalability
- ☐ Availability
- ☐ Reliability
- ☐ Performance
- ☐ Latency
- ☐ Throughput
- ☐ Horizontal vs Vertical Scaling
- ☐ Load Balancing
- ☐ Clustering
- ☐ Caching
- ☐ CDN
- ☐ Proxy
- ☐ Queues
- ☐ CAP Theorem
- ☐ Architectural Styles
- ☐ REST
- ☐ GraphQL
- ☐ gRPC
- ☐ Cloud Design Patterns
- ☐ Long Polling
- ☐ Short Polling
- ☐ WebSockets
- ☐ Server-Sent Events (SSE)
- ☐ Hashing
- ☐ Hashing Algorithms
- ☐ Public Key Cryptography
- ☐ Encryption/Encoding
- ☐ OWASP Top 10

16. PHASE 11 — INTERNSHIP + PLACEMENT PREPARATION

> **When:** VII Semester (placement season) to VIII Semester **Focus:** Placement mode: mocks, applications, offers. Selective Tier 3 only if Tier 1 and 2 are strong.

Part A — DSA

- ☐ Arrays
- ☐ Linked Lists
- ☐ Stack/Queue

☐ Hashing
☐ Binary Search
☐ Sorting
☐ Recursion
☐ Trees
☐ Graphs
☐ Greedy
☐ Dynamic Programming
☐ Advanced DSA
☐ Company-specific DSA
☐ Timed coding practice
☐ Mock coding interviews

Part B — Core CS + Career

☐ DBMS interview questions
☐ OS interview questions
☐ CN interview questions
☐ OOP interview questions
☐ Project explanation
☐ Resume
☐ GitHub
☐ LinkedIn
☐ Aptitude
☐ Mock technical interviews
☐ HR preparation
☐ Internship applications
☐ Placement applications

17. CONCRETE DEVELOPMENT PATH: REACT + NODE.JS + EXPRESS + MYSQL

This expands Phase 9 and Phase 10 Part B into a stack you can actually build with. Spend 3–4 hours per week in regular weeks; none during SEE.

Sem	Stage	Skills
III	Web basics	HTML5, CSS3, Flexbox/Grid, responsive design, JavaScript fundamentals, DOM, Git/GitHub workflow
IV	Advanced JS + React	ES6+, promises, async/await, fetch, closures, array methods; React components, props, state, hooks (useState, useEffect, useContext), React Router, forms, API integration
V	Node + Express	Node basics, npm, Express routing, middleware, REST API design, error handling, environment variables, Postman API testing
V	MySQL	Schema design from your DBMS phases, CRUD, joins, indexes, transactions, mysql2 or Sequelize

Sem	Stage	Skills
V	Auth	JWT, bcrypt password hashing, input validation, CORS, role-based authorization
VI	Deployment + quality	Frontend on Vercel/Netlify, backend on Render/Railway, managed MySQL, .env handling, API tests, UML diagrams, 2–3 design patterns used in real code
VII	System design + security	Scalability, caching, load balancing, queues, REST vs GraphQL, WebSockets, OWASP Top 10, hashing and encryption basics

18. THREE-PROJECT PROGRESSION

Project	Sem	Scope
Project 1: React Frontend App	IV	Example: expense tracker or movie explorer using a public API. Reusable components, routing, responsive UI, deployed with a live link.
Project 2: Full-Stack CRUD with Auth	V	Example: task or notes manager. React frontend, Express REST API, MySQL, JWT login, validation, README with API documentation.
Project 3: Capstone Product	VI	Example: event/placement portal or booking system. Admin/user roles, relational schema with 5+ tables, pagination, search, file upload, deployed, tests for key APIs.

☐ Commit regularly with meaningful messages; recruiters read your GitHub.

☐ Be ready to explain every design choice: why this schema, why JWT, how you would scale it.

☐ Put Project 2 and Project 3 (plus the best of Project 1) on the final resume.

19. PLACEMENT PREPARATION FROM III SEMESTER

Sem	Placement Activities
III	Create GitHub, LeetCode and GeeksforGeeks profiles; draft a 1-page resume; join a coding club or community
IV	Join one hackathon or contest; add Project 1 to resume; create LinkedIn profile
V	Start aptitude practice (30 min/day); first peer mock interviews; attend placement and alumni talks
VI	Apply for summer internships; resume reviewed by seniors; revise DBMS + OS + CN + OOP; LLD and HLD basics
VII	Placement season: weekly mocks, company research, HR preparation, final 1-page resume, referrals and off-campus applications
VIII	Convert offers, complete verification, continue off-campus attempts if needed

20. NON-NEGOTIABLE RETENTION LOOP

☐ Every new DSA topic: learn → implement → easy problem → medium problem → complexity analysis.

☐ Every week: 2–3 old DSA problems.

☐ Every month: one mixed DSA revision session.

☐ Every new Core CS subject: continue solving old DSA problems.

☐ Every semester: revise DBMS + OS + CN + OOP.

☐ Before placements: mixed DSA + Core CS revision, not DSA alone.

Weekly DSA Retention System

Day	Activity
Mon – Thu	New topic: 2 problems per day (one easy/medium, one medium). Write the approach before coding. State time and space complexity for each.
Friday	Re-solve 3–4 old problems from scratch without looking at solutions.
Saturday	Timed session: 2 problems in 60 minutes, or a contest. Review mistakes afterwards.
Sunday	1 hour: update the mistake log, review pattern notes, re-solve failures, plus one Core CS revision (DBMS, OS, CN or OOP).

☐ Spaced re-solving: revisit solved problems on day 1, day 7 and day 30. Star any problem that needed a hint and repeat until clean.

☐ Mistake log: record problem, pattern, why you failed and the fix. Review every Sunday.

☐ Pattern notebook: one page per pattern with C++ template code.

☐ Never read a solution before 30 minutes of honest effort; after reading, close it and re-code from memory.

21. CIE AND SEE EXAM STRATEGY

Academic performance protects placement eligibility, so exam periods are scheduled into the plan.

Period	Typical Timing	What to Do
Regular weeks	Weeks 1–5, 8–10	Full plan: 2 DSA problems/day, 3–4 hrs/week development, weekly cycle as above
CIE-1	~Week 6–7	No new topics. Maintenance: 1 problem/day (max 30 min). Rest of the time goes to exam subjects.
CIE-2	~Week 11–12	Same as CIE-1. Resume the full cycle the week after.
SEE preparation	Last 3–4 weeks	15–20 min/day of DSA revision (re-solve starred problems). Pause development; commit only small fixes.
SEE days	Exam fortnight	Subject study only. Skip DSA on exam days; solve one easy problem on gap days to keep the habit.
Post-SEE break	1–2 weeks	Catch-up window: finish backlog topics, project milestones, mock tests before the next semester.

☐ Never break the streak completely; even one problem keeps pattern memory alive.

☐ Use semester breaks for heavy work: new Tier topics, project milestones, internships.

☐ Keep CGPA high and backlogs at zero; many companies have 6.5–7.5 cut-offs.

☐ If a phase is half-finished when CIE or SEE begins, pause it and resume from the same topic afterwards.

22. FINAL CHECKPOINT FOR EVERY SEMESTER

Tick at least 80% of the boxes before moving to the next semester. Problem counts are targets, not rules.

Checkpoint: III Semester (Phase 0–2)

☐ Write C++ programs without copying; STL comfortable

☐ Estimate time and space complexity of simple code

☐ Arrays, linked list, stack/queue and hashing implemented from scratch

☐ ~100 problems solved
☐ Basic SQL (SELECT, JOIN, GROUP BY) practiced with 20+ queries
☐ HTML/CSS/JS basics and Git workflow in use
☐ GitHub, LeetCode profiles and resume draft created
☐ Passed all CIE/SEE with no backlogs

Checkpoint: IV Semester (Phase 3–4)

☐ Binary search variations and all major sorts understood with complexities
☐ Recursion and backtracking patterns clean (subsets, permutations, N-Queens)
☐ OS basics: processes, threads, scheduling revised
☐ Normalization, ACID, indexes explained in your own words
☐ ~220 cumulative problems; weekly cycle followed for 12+ weeks
☐ Project 1 deployed with a live link and README
☐ Passed all CIE/SEE with no backlogs

Checkpoint: V Semester (Phase 5–6)

☐ Trees, BST, heaps and graph BFS/DFS solved comfortably
☐ Dijkstra, MST and cycle detection understood
☐ OSI, TCP/IP, HTTP, DNS, TLS revised; OS synchronization revised
☐ ~370 cumulative problems; all Tier 1 topics complete
☐ Project 2 (full-stack with JWT auth) deployed
☐ Aptitude practice started; 2+ mock interviews done
☐ Passed all CIE/SEE with no backlogs

Checkpoint: VI Semester (Phase 7–9)

☐ Greedy and DP patterns clean; Tier 2 mostly complete
☐ DBMS + OS + CN consolidation notes finished
☐ ~520 cumulative problems
☐ Project 3 deployed with API tests and clear README
☐ Internship secured or 20+ applications sent
☐ Resume reviewed by 2+ seniors
☐ Passed all CIE/SEE with no backlogs

Checkpoint: VII Semester (Phase 10–11)

☐ System design basics (scaling, caching, load balancing) explainable
☐ OWASP Top 10 and hashing/encryption basics understood
☐ Company-wise sets completed for target companies
☐ 10+ mock interviews (DSA, core CS, HR)
☐ Final 1-page resume and LinkedIn polished
☐ Applied to all relevant on-campus and off-campus openings
☐ At least one offer or final-round progress

Checkpoint: VIII Semester (Phase 11 continued)

☐ Offer(s) confirmed in writing

☐ Final-year project and viva completed

☐ DSA maintenance continued (3–4 problems per week)

☐ Joining formalities and documents ready

☐ Degree requirements and backlog clearance verified

☐ Plan B (off-campus, referrals or higher studies) active if no offer yet

23. FINAL ORDER TO FOLLOW

C++ → Complexity → Basic DSA → Computer Fundamentals → DBMS → Searching/Sorting → OS → Recursion/Backtracking → Trees/Heaps → CN → Graphs → OS/CN Advanced → Greedy/DP → Advanced DSA → Core CS Consolidation → Software Design → Development → Projects → System Design → Security → Placements.

24. SOURCE BASIS

☐ datastructures-and-algorithms.pdf — provided roadmap.sh DSA roadmap

☐ computer-science.pdf — provided roadmap.sh Computer Science roadmap

☐ https://roadmap.sh/datastructures-and-algorithms

☐ https://roadmap.sh/computer-science